

Dossier d'Architecture Technique

Un répéteur particulier propulsé par l'IA pour les six matières écrites du BEPC béninois. Un backend, deux clients : une application web installable et une application Android native.

Concours Afri'Tech Challenge 2026 **Candidat** Prince Kangbode **Document** DAT v1.0

Date 30 août 2026

DÉMONSTRATION repetia.vercel.app	API PUBLIQUE repetia-api.onrender.com
CODE SOURCE github.com/Nounagnon02/repetia	APPLICATION ANDROID Expo — build EAS, APK sur demande

1 — PRÉSENTATION DE LA SOLUTION

Objectifs, usages et publics cibles

Le problème

Au Bénin, le répéteur particulier est la norme pour préparer le BEPC — et il coûte entre 5 000 et 15 000 FCFA par mois. Beaucoup de familles ne peuvent pas se le permettre. L'élève se retrouve seul face à un exercice qu'il ne comprend pas, avec un manuel qui donne la réponse mais jamais la démarche.

Les ressources en ligne existantes échouent sur trois points concrets : elles ne suivent pas le programme béninois, elles supposent une connexion stable et un forfait de données confortable, et elles donnent des corrigés au lieu d'expliquer.

Objectifs

- Mettre un répéteur dans la poche de chaque élève, gratuitement et sans compte.
- Expliquer la **démarche**, jamais seulement la réponse.
- Rester utilisable sur un téléphone d'entrée de gamme et un forfait de données limité.
- Couvrir le programme béninois, pas un programme générique importé.

Publics cibles

PUBLIC	USAGE
Élèves de 3 ^e préparant le BEPC	Révision autonome le soir et le week-end, sur téléphone
Parents sans moyens pour un répétiteur	Suivre la progression de l'enfant par thème
Enseignants et écoles	Exercices d'appoint, repérage des thèmes fragiles d'une classe

Fonctionnalités livrées

RÉF	FONCTIONNALITÉ	ÉTAT
F1	Sélection de la matière (6), du thème (46) et de la difficulté	Livré
F2	Génération d'un exercice par l'IA, adapté au thème et au niveau	Livré
F3	Saisie de la réponse et correction, formes équivalentes acceptées	Livré
F4	Explication pas à pas après chaque tentative	Livré
F5	Chat répétiteur, avec le contexte de l'exercice en cours	Livré
F6	Suivi de progression : taux global et maîtrise par thème	Livré
F7	Recommandation du thème à revoir en priorité	Livré

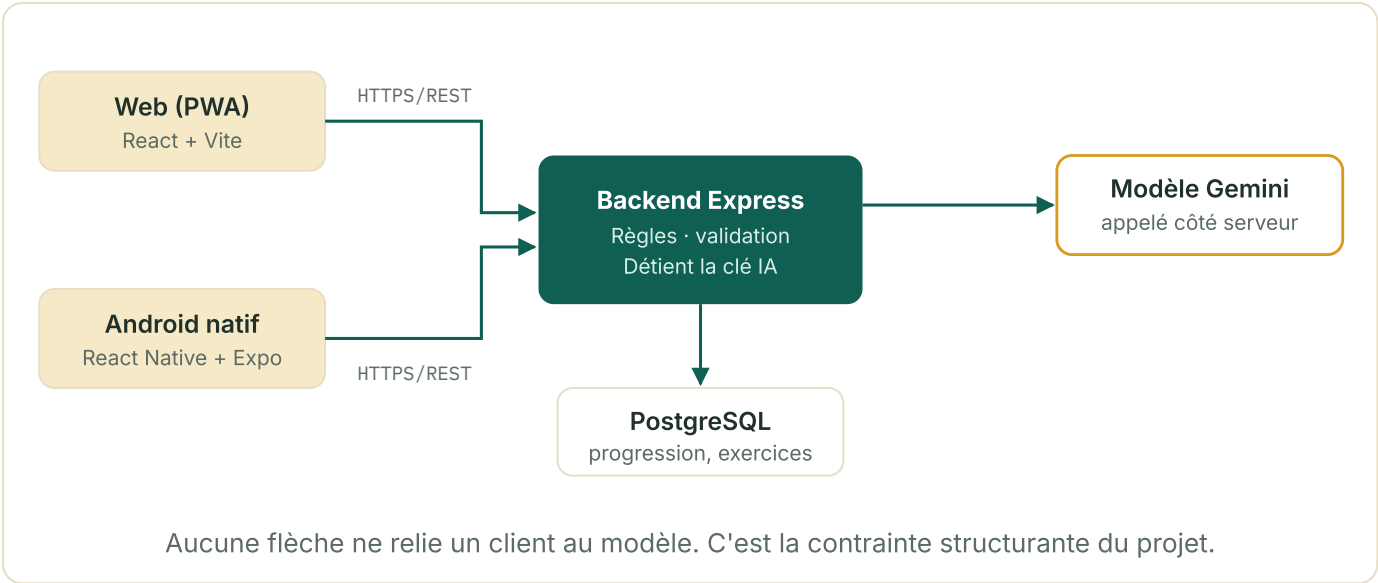
Périmètre couvert

MATIÈRE	THÈMES	EXEMPLES
Mathématiques	8	Thalès, Pythagore, équations, racines carrées
Physique-Chimie-Technologie	8	Loi d'Ohm, forces, réactions chimiques
Sciences de la Vie et de la Terre	8	Digestion, génétique, écosystèmes
Français	7	Grammaire, conjugaison, figures de style
Anglais	7	Tenses, reported speech, compréhension
Histoire-Géographie	8	Indépendances, climats et économie du Bénin

Six matières, 46 thèmes. Le produit est en ligne et testable dès aujourd'hui.

2 — ARCHITECTURE GLOBALE

Un backend souverain, deux clients



Composants

COMPOSANT	RÔLE	HÉBERGEMENT
Frontend web	PWA installable, 4 écrans, cache hors-ligne	Vercel (CDN mondial)
Frontend mobile	Application Android native, mêmes écrans	APK / Play Store via EAS
Backend	API REST, règles métier, seul appelant du modèle	Render — région Francfort
Base de données	Élèves anonymes, exercices, tentatives, progression	PostgreSQL 16 managé, non exposée
Service d'IA	Génération, correction, chat	Google Gemini, appelé serveur à serveur

Modèle de données

Six entités, reliées de l'élève à sa progression par thème :

ENTITÉ	CONTENU	RELATIONS
User	Identifiant anonyme, niveau	1 → n tentatives, progressions
Matiere	Code, libellé, niveau, ordre	1 → n thèmes
Theme	Libellé, ordre pédagogique	n → 1 matière
Exercice	Énoncé, solution, explication, provenance	n → 1 thème
Tentative	Réponse de l'élève, verdict	n → 1 élève, 1 exercice
Progression	Score de maîtrise, compteurs	unique par (élève, thème)

Contrat d'API

MÉTHODE	ROUTE	RÔLE
GET	/api/matieres	Matières du niveau
GET	/api/matieres/:id/themes	Thèmes, triés par progression pédagogique
POST	/api/exercices/generer	Génère un exercice — ne renvoie ni solution ni explication
POST	/api/tentatives	Corrige, explique, met à jour la maîtrise
GET	/api/progression	Global, par thème, et thème à revoir
POST	/api/chat	Question libre, avec l'exercice en contexte
GET	/health	État du serveur, de la base et de la configuration IA

La solution reste au serveur jusqu'à la tentative

`/api/exercices/generer` renvoie l'énoncé, et rien d'autre. La solution est écrite en base mais jamais transmise avant que l'élève ait soumis sa réponse — sinon il suffirait d'ouvrir les outils de développement pour lire le corrigé, et l'outil perdrait tout intérêt pédagogique. Un test verrouille ce comportement.

3 – CHOIX TECHNOLOGIQUES

Peu de dépendances, chacune justifiée

COUCHE	CHOIX	VERSION	POURQUOI
Langage	TypeScript	5.9	Un seul langage du serveur au mobile ; le contrat d'API est typé des deux côtés
Backend	Node.js · Express	5.2	Minimal, sans magie, adapté à une petite équipe
ORM	Prisma	5.22	Schéma unique, migration SQLite → PostgreSQL sans réécrire une requête
Base de données	SQLite (dev) · PostgreSQL 16 (prod)	16	Zéro installation pour développer, base managée en production
Validation	Zod	4.5	Valide les entrées HTTP et les sorties du modèle avec le même outil
Sécurité	Helmet · CORS · express-rate-limit	8.x	En-têtes durcis, origines verrouillées, quota par élève
Frontend web	React · Vite · Tailwind	React 19.2	Bundle léger, PWA installable, build en moins d'une seconde
Frontend mobile	React Native · Expo · Expo Router	SDK 57	Un seul socle React, build Android sans machine dédiée
Styles mobiles	NativeWind	4.2	Les jetons de design du web réutilisés tels quels sur mobile
IA	Google Gemini (@google/genai)	2.19	Palier gratuit exploitable, latence acceptable, bon français
Tests	Jest · Vitest · Testing Library · jest-expo	–	Une commande unique couvre les trois espaces

Neuf dépendances de production côté serveur

C'est un choix, pas une limite. Chaque paquet ajouté est une surface de faille, une mise à jour à suivre et un risque de rupture. Le rendu du texte enrichi, par exemple, est écrit à la main plutôt qu'apporté par une bibliothèque Markdown : quelques dizaines de lignes contre plusieurs dizaines de kilo-octets envoyés à chaque élève.

Outils de développement

- **Git** — 34 commits en *Conventional Commits*, historique lisible et rattachable à chaque décision.
- **TypeScript strict côté clients** — le typecheck fait partie de la définition du « terminé ».
- **Prisma CLI** — génération du client, application du schéma, seed idempotent.
- **EAS Build** — production de l'APK Android sans SDK Android installé localement.
- **Scripts de génération** — le logo et les icônes des deux clients sont dérivés d'une géométrie unique ; ils ne peuvent pas diverger.

4 — SCALABILITÉ & PERFORMANCE

Le goulot n'est pas le serveur, c'est le modèle

Le backend est **sans état** : toute la session tient dans l'en-tête `X-User-Id` et en base. Deux conséquences directes : on peut multiplier les instances derrière un répartiteur de charge sans configuration particulière, et le redémarrage d'une instance ne perd aucune session.

Où passe le temps

OPÉRATION	TEMPS MESURÉ	FACTEUR LIMITANT
Catalogue (matières, thèmes)	< 200 ms	Base de données
Progression	< 200 ms	Base de données
Génération d'un exercice	10 – 17 s	Modèle d'IA
Correction d'une tentative	5 – 19 s	Modèle d'IA
Réponse du chat	7 – 14 s	Modèle d'IA

Les mesures proviennent de la suite d'intégration exécutée contre l'instance de production. Tout ce qui ne dépend pas du modèle répond en moins d'un cinquième de seconde ; la montée en charge se joue donc entièrement sur les appels d'IA.

Stratégie de cache

NIVEAU	CONTENU	STRATÉGIE
Service worker (web)	Coquille de l'application, polices	<i>Precache</i> — 7 entrées, mise à jour au déploiement
Service worker (web)	<code>GET /api/matieres</code> , <code>/api/progression</code>	<i>NetworkFirst</i> , bascule après 5 s, 7 jours de rétention
Stockage local (mobile)	Thèmes, progression, 10 derniers exercices	Lecture immédiate, rafraîchie en arrière-plan
Base de données	Exercices générés	Conservés et réutilisables — un exercice payé une fois sert plusieurs élèves

Maîtrise de la charge et du coût

- **Quota par élève** — 20 appels IA par minute, réglable par variable d'environnement.
- **Corps de requête plafonné** à 64 ko, longueurs de champs bornées par schéma.
- **Dégradation avant rupture** — quand le modèle sature, l'application sert la banque locale plutôt que d'échouer (voir §8).
- **Bundle web à 97 Ko compressés**, très en dessous de la cible de 500 Ko : moins de données facturées à la famille, chargement rapide en 3G.

Montée en charge envisagée

1. **Aujourd'hui** — une instance, base managée, palier gratuit. Suffisant pour la démonstration et les premiers utilisateurs.
2. **Croissance** — instances multiples derrière le répartiteur de l'hébergeur ; le backend étant sans état, aucune modification de code.
3. **Échelle** — mise en cache des exercices par (thème, difficulté) pour servir sans appeler le modèle, et file d'attente pour lisser les pics.

5 — INTEROPÉRABILITÉ

Une API REST que n'importe quel client peut consommer

Le backend ne connaît pas ses clients. Il expose une API REST sur HTTPS, en JSON, avec des codes de statut standards — et c'est précisément ce qui a permis d'ajouter une application Android à un produit conçu pour le web **sans modifier une seule route**.

STANDARD	APPLICATION
REST sur HTTPS	Routes orientées ressources, verbes HTTP respectés
JSON	Échanges dans les deux sens, encodage UTF-8
Codes de statut HTTP	200, 400 (entrée invalide), 401 (identité), 404, 429 (quota), 503 (IA indisponible)
CORS	Origines déclarées explicitement en production
En-têtes de sécurité	Helmet — <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , HSTS...
Sonde de santé	<code>GET /health</code> — état du serveur, de la base et de la configuration IA

La preuve par l'usage

Le client Android a été ajouté après coup. L'API n'a pas changé. De même, l'extension d'une à six matières n'a demandé **aucune modification de route** : le modèle de données portait matière et niveau dès la première version.

Intégrations tierces

- **Google Gemini** — appelé serveur à serveur, avec chaîne de repli sur un second modèle puis sur une banque locale.
- **Ouverture prévue** — l'API se prête telle quelle à une intégration par un établissement (tableau de bord de classe) ou à un client SMS/USSD pour les élèves sans smartphone.

6 — MÉTHODOLOGIE DE PROJET

Cycle de vie, versioning et environnements

Cycle de développement

Le projet a été mené par incréments courts, chacun terminé par la même séquence : **typecheck** → **tests** → **vérification dans l'application réelle** → **commit**. Une fonctionnalité n'est déclarée finie que lorsque les trois passent.

ÉTAPE	COMMANDE	CRITÈRE
Vérification des types	<code>npm run typecheck</code>	0 erreur sur les trois espaces
Tests	<code>npm test</code>	140 tests au vert
Compilation	<code>npm run build</code>	Backend et web compilent
Vérification réelle	<code>npm run dev</code>	Parcours joué dans l'application
Publication	<code>git push</code>	Déploiement automatique

Versioning

- **Git**, dépôt public : `github.com/Nounagnon02/repetia`
- **Conventional Commits** — `feat`, `fix`, `test`, `docs`, `chore`, avec le périmètre entre parenthèses.
- **Commits atomiques** : un commit = une décision, dont le message explique le *pourquoi*, pas seulement le *quoi*.
- **Aucun secret dans l'historique** — vérifié sur l'intégralité des 34 commits.

Environnements

	DÉVELOPPEMENT	TEST	PRODUCTION
Base	SQLite locale	SQLite dédiée, recrée à chaque exécution	PostgreSQL 16 managé
Modèle d'IA	Clé réelle	Simulé — ni clé, ni réseau, ni coût	Clé réelle, chaîne à deux modèles
Origines CORS	Toutes	Aucune	Le domaine du frontend uniquement
Quota IA	20 / min	5 / min, pour tester le rejet	20 / min par élève

La base de test est physiquement séparée

Une première version des tests vidait la base de développement à chaque exécution. Elle utilise désormais une base dédiée, recrée avant chaque campagne. Un incident réel, corrigé et verrouillé par la configuration.

Configuration

Aucun secret n'est écrit dans le code. Tout passe par des variables d'environnement, dont un `.env.example` commenté est versionné. Les `.env` réels sont exclus du dépôt. La sonde `/health` indique si la clé d'IA est bien vue par le serveur — sans jamais l'exposer.

Le moins de données possible, la clé hors de portée

La clé d'IA ne descend jamais dans le client

Une clé embarquée dans une application mobile est extractible en quelques minutes : il suffit de décompiler l'APK. Elle serait alors utilisée par des tiers, aux frais du projet, jusqu'à épuisement du quota — et l'application tomberait pour tous les élèves. Un seul fichier importe le SDK d'IA. Vérification reproductible sur le code publié :

```
grep -r "LLM_API_KEY\|GoogleGenAI" frontend/src mobile/src
```

Cette commande ne renvoie rien. L'audit du bundle Android exporté ne renvoie rien non plus.

Gestion des accès

- **Identifiant anonyme** transmis dans l'en-tête `X-User-Id`, validé par expression régulière avant toute requête en base.
- **Routes de lecture publiques** (catalogue) et **routes identifiées** pour tout ce qui touche à l'élève.
- **Cloisonnement** — un élève ne peut lire que sa propre progression ; l'identifiant sert de clé de partitionnement dans chaque requête.

Données personnelles et protection des mineurs

Aucun compte, aucune donnée personnelle

L'élève ne fournit ni nom, ni email, ni téléphone, ni mot de passe. Un identifiant aléatoire est généré au premier lancement et conservé sur l'appareil. Le serveur n'associe à cet identifiant que des données scolaires : thèmes travaillés, réponses, scores. Le public visé étant mineur, c'est la position la plus sûre — **il n'y a rien à fuiter**, et rien qui relève d'un traitement de données sensibles.

Cette conception place le service très en deçà des obligations habituelles en matière de données personnelles : pas de collecte, pas de profilage, pas de transfert à des tiers. Les requêtes envoyées au modèle ne contiennent que l'énoncé, la réponse de l'élève et le thème — jamais d'identifiant ni de donnée nominative.

Défenses applicatives

RISQUE	MESURE
Injection dans le prompt	La difficulté passe par une liste blanche ; longueurs bornées par schéma
Entrées malformées	Validation Zod sur corps, requête et paramètres — 400 explicite, jamais 500
Abus de quota	Limitation par élève, pas par IP (voir §9)
Requêtes hors origine	CORS restreint au domaine du frontend en production
En-têtes et clickjacking	Helmet
Charge utile excessive	Corps limité à 64 ko
Fuite de secrets	<code>.env</code> git-ignoré ; historique complet audité
Transport	HTTPS de bout en bout, certificats gérés par les hébergeurs

Ce qui se passe quand quelque chose casse

Robustesse du service d'IA

Un modèle de langage tombe, sature, ou renvoie du texte inexploitable. Sur un produit destiné à des élèves, une panne d'IA ne doit pas devenir une page blanche. Quatre niveaux de repli s'enchaînent avant tout échec visible :

- 1 Nettoyage de la réponse**
Retrait des balises Markdown, extraction du premier objet JSON dans un texte bavard.
- 2 Validation par schéma**
Les trois champs doivent être présents *et* non vides. Un `{}` est un JSON parfaitement valide : sans cette étape il traversait le service et faisait échouer l'écriture en base par une erreur 500. Incident réel, corrigé.
- 3 Second modèle**
Le quota gratuit se compte par modèle et par jour. Quand le principal est épuisé, un modèle allégé prend le relais — l'élève garde une vraie explication.
- 4 Banque locale**
42 exercices rédigés à la main. Les 8 thèmes de mathématiques sont couverts au thème près ; chaque autre matière dispose de trois exercices justes et de son niveau. Marqués `source: "banque"`, ce qui permet de mesurer la fréquence du repli.

La correction dispose du même filet. Le chat renvoie une erreur `503` explicite : il n'existe pas de repli honnête à une question libre, et faire passer un message d'erreur pour une réponse du répétiteur serait pire que d'admettre la panne.

Qualité et non-régression

140 tests automatisés, 0 échec	68 backend — API, IA, texte	61 mobile — parcours, cache	11 web — parcours clé
--	---------------------------------------	---------------------------------------	---------------------------------

Une commande unique, `npm test`, couvre les trois espaces. Le service d'IA est systématiquement simulé : la suite tourne sans clé, sans réseau et sans coût. S'y ajoute une suite d'intégration optionnelle qui valide le contrat contre le *vrai* backend déployé, modèle compris — passée au vert sur l'instance de production.

Ce que les tests verrouillent, entre autres :

- La génération ne renvoie jamais la solution ni l'explication.
- Une réponse d'IA malformée déclenche le repli sans planter.
- Une panne du chat produit un `503` et une vraie erreur affichée, jamais une fausse réponse.
- La validation des entrées, le quota, le calcul du score de maîtrise.

Stratégie de mise à jour

- **Déploiement continu** — un `git push` déclenche la compilation, l'application du schéma et la mise en ligne. Aucune commande manuelle après un déploiement : le catalogue s'installe seul au démarrage.
- **Sonde de santé** — l'hébergeur ne bascule sur la nouvelle version que si `/health` répond.
- **Versions figées** — `package-lock.json` versionné : une réinstallation produit exactement le même arbre de dépendances.
- **Montées de version** — dépendances peu nombreuses et à jour ; les paliers majeurs (Node, Expo) se traitent un par un, la suite de tests servant de garde-fou.

Évolutivité fonctionnelle

Le modèle de données a été conçu pour l'extension, et cela a été vérifié en pratique :

- **Ajouter une matière** — une entrée dans le catalogue. Les cinq matières ajoutées après les mathématiques n'ont demandé aucune modification de l'API.
- **Ajouter un niveau** (BAC) — le champ `niveau` existe déjà sur la matière ; c'est un seed.
- **Changer de modèle d'IA** — le nom du modèle est une variable d'environnement ; la chaîne de repli en accepte plusieurs.
- **Ajouter un client** — l'API est publique et documentée ; le client Android en est la démonstration.

9 — ANCRAGE LOCAL ET LIMITES

Des décisions dictées par le terrain béninois

La limitation de débit compte par élève, pas par adresse IP

Une classe entière ou un cybercafé partage une seule adresse IP publique. Limiter par IP — le réglage par défaut de toutes les bibliothèques — aurait bloqué tout un établissement dès que quelques élèves travaillent ensemble. Le compteur porte donc sur l'identifiant de l'élève.

Écrire les mathématiques comme au tableau

Le modèle produit spontanément du LaTeX — l'élève lisait `\sqrt{45}` tel quel. Embarquer un moteur LaTeX aurait coûté plusieurs centaines de kilo-octets sur des téléphones d'entrée de gamme, pour afficher ce qu'un élève lit déjà au tableau. Le serveur convertit donc en Unicode : `√45`, `x²`, `3 × 5`, `(MN) ∥ (BC)`. Les consignes ne s'appliquent qu'aux matières scientifiques : les imposer en anglais n'aurait aucun sens.

Fonctionnement dégradé

- La coquille de l'application et les thèmes sont mis en cache : elle s'ouvre sans réseau.

- Les **dix derniers exercices** travaillés, corrections comprises, restent consultables hors connexion.
- Sans clé d'API configurée, le serveur démarre quand même et sert la banque locale.

Accessibilité

Interface intégralement en français, langage simple. Contrastes conformes AA. Chaque bouton icône porte un libellé accessible, les groupes de choix sont navigables au clavier, les chargements sont annoncés aux lecteurs d'écran et les erreurs signalées comme telles. Mise en page utilisable dès 320 pixels de large.

Limites assumées

Les dire est plus utile que les taire — elles conditionnent la suite du projet.

LIMITE	CONSÉQUENCE	TRAITEMENT PRÉVU
Mise en veille de l'offre gratuite	La première visite après 15 min d'inactivité prend ~30 s	Offre payante ou tâche de réveil planifiée
Quota du palier gratuit de l'IA	Sous forte affluence, l'application sert la banque locale	Activation de la facturation à l'usage
Un seul niveau	BEPC uniquement, les six matières écrites	Le modèle porte déjà le niveau : ajouter le BAC est un seed
Thèmes non validés officiellement	Découpage établi d'après le programme, à confirmer	Relecture par des enseignants du secondaire
Pas de migrations versionnées	Suffisant sans données de production à préserver	Migrations avant la mise en service réelle
Contenu non validé par des enseignants	La pertinence repose sur le modèle et sur la banque	Panel de professeurs de collège

Suite envisagée

- Validation des exercices générés par un panel d'enseignants béninois.
- Extension au BAC — l'architecture le prévoit déjà.
- Mode « examen blanc » chronométré sur sujets d'annales.
- Tableau de bord de classe pour les enseignants, via l'API existante.
- Explications en langues locales, pour les élèves qui butent d'abord sur le français.

ANNEXES

Déploiement et vérifications

Infrastructure en production

COMPOSANT	HÉBERGEMENT	ADRESSE
Application web	Vercel	repetia.vercel.app
API	Render — région Francfort	repetia-api.onrender.com
Base de données	PostgreSQL 16 managé	interne, non exposée
Application Android	EAS Build	APK / AAB

Francfort est la région la plus proche de l'Afrique de l'Ouest chez cet hébergeur. Le schéma est appliqué au build ; le catalogue s'installe au démarrage du serveur.

Score de maîtrise — formule

Après chaque tentative, le score du thème suit une moyenne mobile exponentielle ($\alpha = 0,3$) :

$$\text{nouveau_score} = \text{ancien_score} \times 0,7 + (100 \text{ si juste, sinon } 0) \times 0,3$$

Les tentatives récentes pèsent donc davantage — un élève qui progresse voit son score remonter au lieu d'être puni indéfiniment par ses débuts.

Vérifier soi-même

VÉRIFICATION	COMMANDE
Santé du service	<code>curl https://repetia-api.onrender.com/health</code>
Catalogue	<code>curl "https://repetia-api.onrender.com/api/matieres?niveau=BEPC"</code>
Aucune clé côté client	<code>grep -r "LLM_API_KEY" frontend/src mobile/src</code>
Suite de tests	<code>npm install && npm test</code>

« *Notre problème, ma solution.* » Le code, les tests et la présente note sont publics : github.com/Nounagnon02/repetia.